

WebAssembly Component Model Stability Status

https://gemini.google.com/app/990a1ffb0da82153

User prompt: Are WebAssembly component stable?

Response: **Yes, the WebAssembly Component Model is stable for production use.**

The stabilization reached a major milestone with the release of **WASI 0.2 (WASI Preview 2)**, which formally established the Component Model as the foundation for the Wasm system interface.

Key Stability Milestones

- Interface Contract (WIT):** Wasm Interface Type (WIT) definitions and binary specs under WASI 0.2.x are backward-compatible. Code compiled against WASI 0.2 target definitions remains stable.
- Runtime Support:** Major runtimes—most notably **Wasmtime** (Bytecode Alliance) and **Wasmer**—fully support binary loading, execution, and composition of Wasm components using WASI 0.2+.
- Toolchain Integration:** Native language toolchains (such as Rust's `cargo-component`, Go's `tinygo`, and Python/JS guest bindings) provide production-ready tooling to build, bind, and package component binaries.

What to Keep in Mind

Area	Current Status
WASI 0.2 / Preview 2	Stable. Includes standard interfaces for HTTP (<code>wasi-http</code>), CLI, clocks, random, and filesystem access.
Component Composition	Stable. Dynamic linking and multi-language capability linking (e.g., calling a Rust component from a Python host) via WIT contracts.
WASI 0.3 (Preview 3)	In Development. Focuses on native <code>async/await</code> primitives across component boundaries.

Recommendation

If you are starting a new system using standard WASI 0.2 interfaces or building portable microservices/plugin architectures, you can safely use the Component Model in production. If your architecture relies heavily on non-blocking `async` operations between component boundaries, be aware that `async` conventions are transitioning toward WASI 0.3 native `async` support.

Sources

- [The WebAssembly Component Model: Introduction](#)
- [WASI and the WebAssembly Component Model: Current Status](#)
- [The Rise of WASM-Native Runtimes for AI Tools](#)

User prompt: What is the relationship of components between WIT, WAT and WASM files?

Response: WIT, WAT, and WASM represent three distinct layers of abstraction: **WIT** is the high-level interface contract, **WAT** is the human-readable low-level assembly representation, and **WASM** is the binary output executed by a runtime.

Core Roles

Format	Full Name	Level	Primary Purpose
WIT	Wasm Interface Type	High-Level IDL	Defines high-level types, imports, and exports (e.g., <code>string</code> , <code>list</code> , <code>records</code>) between components across language boundaries.
WAT	Wasm Text Format	Low-Level Text	Human-readable representation of raw WebAssembly instructions, linear memory, and basic types (<code>i32</code> , <code>f64</code>).
WASM	WebAssembly Binary	Low-Level Machine Code	The compiled, compact binary executed directly by Wasm runtimes (like Wasmtime or V8).

How They Work Together

- Design & Contract (WIT):** You write or generate a `.wit` file defining what a component exposes or needs (e.g., `func greet(name: string) -> string`).
- Implementation:** Tools (`wit-bindgen`, `cargo-component`) use the WIT contract to generate language-specific boilerplate (C#, Rust, Go), which you implement.
- Compilation (WASM):** The source code compiles down to a `.wasm` binary containing both the execution bytecode and embedded WIT metadata sections.
- Inspection & Debugging (WAT):** Tools like `wasm2wat` convert the `.wasm` binary into human-readable `.wat` text so developers can inspect the low-level stack instructions.

Analogies to Traditional Software

- **WIT** is like a **C Header file** (`.h`) or a **Protobuf schema** — it describes *what* can be called without implementation details.
- **WAT** is like **x86 Assembly** (`.s`) — readable assembly instructions representing the underlying execution model.
- **WASM** is like a **DLL, shared object** (`.so`), or **Java bytecode** (`.class`) — the actual binary artifact consumed by execution engines.

User prompt: How is WIT components rendered in a WAT file?

Response: When a WebAssembly component compiled from a WIT contract is disassembled into its textual format, the resulting WAT file transitions from a standard `(module ...)` to a high-level `(component ...)` definition.

The WebAssembly Component Model extends the core WebAssembly Text Format with explicit component-level abstractions. The high-level types, imports, and exports declared in a `.wit` file map directly into specific WAT S-expressions.

Key Structural Mapping: WIT to WAT

WIT Concept	Rendered WAT S-Expression	Purpose in WAT
interface / world	<code>(component ...)</code>	The top-level root node wrapping nested modules, canonical instances, and interface definitions.
High-level Types	<code>(type (func ...)) & (type (defined ...))</code>	Defines component-level types (e.g., <code>string</code> , <code>list</code> , <code>records</code>) that sit above core Wasm primitives (<code>i32</code> , <code>f64</code>).
import / export	<code>(import "..." ...) / (export "..." ...)</code>	Declares imported or exported functions, interfaces, and instances by their WIT identifier paths.
Core Execution Code	<code>(core module ...)</code>	Houses the compiled Wasm bytecode (functions, linear memory, stack ops) generated by language compilers like Rust or C.
ABI Translation	<code>(canon lift ...) & (canon lower ...)</code>	Canonical ABI glue layer mapping core functions and linear memory into high-level component types.

Concrete Code Example

1. The WIT Definition (`greeter.wit`)

```
package local:demo;

interface api {
  greet: func(name: string) -> string;
}

world app {
  export api;
}
```

2. How it Renders in Component WAT (`greeter.wat`)

When converted to WAT using tools like `wasm-tools print`, the file structures the embedded core module and canonical bindings:

```
(component
  ;; 1. Core WebAssembly module containing actual compiled execution logic
  (core module $main_engine
    (memory (;0;) 1)
    (func $core_greet (param i32 i32 i32)) ;; Low-level string ABI: (ptr, len, ret_ptr)
    (export "memory" (memory 0))
    (export "greet_raw" (func $core_greet))
  )

  ;; 2. Instantiate the core module inside the component
  (core instance $main_inst (instantiate $main_engine))

  ;; 3. Component Type Definitions (derived from WIT string types)
  (type $greet_type (func (param "name" string) (result string)))

  ;; 4. Canonical Lift: Converts low-level pointers/memory into WIT high-level types
  (func $lifted_greet (type $greet_type)
    (canon lift
      (core func $main_inst "greet_raw")
      (memory $main_inst "memory")
    )
  )

  ;; 5. Interface Instance Exporting the WIT Function
  (instance $api_instance
    (export "greet" (func $lifted_greet))
  )
)
```

```
;; 6. Export the WIT Interface under its package identifier
(export "local:demo/api" (instance $api_instance))
)
```

Architectural Takeaways

1. **Self-Describing Encapsulation:** Unlike raw Wasm modules—which require host JavaScript or external glue code to translate complex types—a component WAT contains both the executable bytecode (`core module`) and the component glue (`canon lift/lower`) in a single readable tree.
2. **Canonical ABI:** The fundamental gap between WIT (`string`) and WAT (`i32` pointer in linear memory) is bridged inside the component text format using (`canon lift ...`) for exports and (`canon lower ...`) for imports.

Sources

1. [wit-component - crates.io: Rust Package Registry](#)
2. [Frequently Asked Questions \(FAQ\) - The WebAssembly Component](#)
3. [Understanding WebAssembly text format - MDN Web Docs](#)

User prompt: Did this require a change in the format of WASM binary file to represent components?

Response: **Yes, the WebAssembly Component Model required a direct update to the WASM binary format specification.**

Although both use the `.wasm` file extension, a **Component WASM** binary is structurally distinct from a traditional **Core WASM** binary.

How the Binary Format Changed

A traditional Wasm runtime checks the header bytes of a file before parsing. The Component Model modified the binary header layout and introduced component-specific section wrappers:

1. Header Discrimination (`Layer Bytes`)

Both formats share the same initial 4-byte magic string (`\0asm / 0x00 0x61 0x73 0x6D`). However, the 4 bytes that follow—which previously held the Core version—were re-interpreted into two 16-bit fields (`Version + Layer`) to distinguish them:

- **Core Wasm Module:** `0x00 0x61 0x73 0x6D | 0x01 0x00 (Version 1) 0x00 0x00 (Layer 0)`
- **Component Wasm:** `0x00 0x61 0x73 0x6D | 0x0d 0x00 (Version) 0x01 0x00 (Layer 1)`

This allows engines like Wasmtime or V8 to recognize within the first 8 bytes whether they are inspecting a core module or a component.

2. New Component-Level Sections

A core WASM binary contains linear sections like `type` (1), `import` (2), `function` (3), `code` (10), and `memory` (5).

The Component Model introduced a set of **component section ID tags**. These include:

- **Embedded Core Modules (`0x01`):** A component binary can embed one or more complete Core `.wasm` binaries directly inside itself as nested sections.
- **Component Types (`0x02`):** Encodes WIT-level data types (`record`, `variant`, `list`, `string`).
- **Canonical Functions (`0x0A`):** Encodes the `canon lift` and `canon lower` instructions that map linear-memory functions to interface types.
- **Component Instances & Exports (`0x05`, `0x07`):** Encodes high-level instances and namespaced export strings (e.g., `wasi:cli/run@0.2.0`).

Backward Compatibility Strategy

To ensure traditional Wasm runtimes don't crash when encountering a component binary:

1. **Nested Containment:** Core Wasm modules remain untouched inside the component. A component literally wraps existing `.wasm` bytecode inside its own component-level sections.
2. **Adapter Tooling:** If you feed a Component `.wasm` file to an older Wasm engine that only supports Core Wasm (Layer 0), the runtime will immediately reject the header `Layer` value rather than executing corrupted memory instructions. Tools like `wasm-tools` can unwrap components back into standard core modules if backward compatibility is needed.

Sources

1. [Frequently Asked Questions \(FAQ\) - The WebAssembly Component](#)
2. [component-model/design/mvp/Binary.md at main - GitHub](#)
3. [component-model/design/mvp/Explainer.md at main - GitHub](#)
4. [component-model/design/mvp/Linking.md at main - GitHub](#)

User prompt: Where can I find the description of the binary format?

Response: The official specification for the WebAssembly Component Model binary format is maintained in the **W3C WebAssembly**

Community Group repository on GitHub.

Primary Specification Locations

- **Binary Format Explainer:** [WebAssembly/component-model/design/mvp/Binary.md](#)
 - This is the exact normative document detailing the binary structure. It covers section opcodes, preamble header semantics, LEB128 encoding rules, and component-level AST definitions.
- **Component Model Design Repository:** [WebAssembly/component-model](#)
 - The primary W3C repository hosting all sub-specifications, including the Canonical ABI ([CanonicalABI.mc](#)), WebAssembly Interface Types ([WIT.md](#)), and linking models.

Core Binary Format Summary

Specification Element	Value / Rule
Magic Number	<code>\0asm(0x00 0x61 0x73 0x6D)</code>
Layer & Version Field	<code>0x0D 0x00</code> (Version) + <code>0x01 0x00</code> (Layer 1 = Component)
Core Module Section (0x01)	Inlines nested Core Wasm binaries (<code>Layer 0</code>) directly inside the component.
Component Types (0x02)	Encodes high-level WIT types (<code>record</code> , <code>variant</code> , <code>list</code> , <code>string</code>).
Canonical Lift/Lower (0x0A)	Encodes ABI glue mapping linear-memory values to component interfaces.

Interactive Exploration Tools

If you want to inspect or disassemble the binary layout of actual component files locally, the **Bytecode Alliance** provides CLI tools:

- `wasm-tools print file.wasm`: Disassembles binary component sections into WAT text.
- `wasm-tools objdump file.wasm`: Displays section headers, byte offsets, and raw section opcodes within a `.wasm` file.

Sources

1. [api-evangelist/component-model - GitHub](#)
2. [component-model/design/mvp/Binary.md at main - GitHub](#)
3. [Frequently Asked Questions \(FAQ\) - The WebAssembly Component](#)

User prompt: Explain better and more in detail the (canon lift ...) & (canon lower ...)

Response: The `canon lift` and `canon lower` instructions form the **Canonical ABI** (Application Binary Interface) glue layer in the WebAssembly Component Model. Their job is to translate data between the high-level world of components (which understand complex types like `string`, `list`, `variant`, and `record`) and the low-level world of core WebAssembly (which only understands primitive numbers like `i32`, `f64`, and raw linear memory pointers).

The Fundamental Problem They Solve

A core WebAssembly module has no concept of a "string" or a "struct"—it only understands basic numeric types (`i32`, `i64`, `f32`, `f64`).

If core Wasm wants to send the string `"hello"` to another module:

1. Core Wasm represents `"hello"` as a memory pointer (e.g., address `1024`) and a length (`5`).
2. The receiving module has its own completely separate, isolated linear memory space, so address `1024` in Module A means nothing to Module B.

`canon lift` and `canon lower` bridge this gap by acting as automated data marshaling functions.

`canon lower`: High-Level to Low-Level

`canon lower` takes a **high-level component function** and converts ("lowers") it into a **low-level core Wasm function** so that core Wasm execution logic can invoke it.

- **Direction:** Component Interface → Core Wasm Logic
- **Use Case:** When core Wasm needs to **import and call** an external WIT function.

What happens step-by-step:

1. Core Wasm calls a lowered function, passing basic primitives (e.g., passing `i32` pointer `1024` and `i32` length `5` representing a string).
2. `canon lower` reads the bytes directly out of the caller's linear memory.
3. It converts those raw bytes into a high-level Component Model type (e.g., an abstract UTF-8 `string`).
4. It passes that abstract high-level value across the component boundary to the imported target.

canon lift: Low-Level to High-Level

canon lift does the exact opposite: it takes a **low-level core Wasm function** and wraps ("lifts") it into a **high-level component function** that can be exported to other components.

- **Direction:** Core Wasm Logic → Component Interface
- **Use Case:** When core Wasm wants to **export** an internal function as a WIT interface function.

What happens step-by-step:

1. An outside caller invokes the lifted component function, passing high-level values (e.g., a high-level `string` object).
2. `canon lift` allocates memory inside the destination core Wasm module's linear memory (using a helper function named `realloc`).
3. It writes the high-level string data directly into that allocated space.
4. It passes the resulting memory address (`i32` pointer) and string length (`i32` length) to the internal core Wasm function.

Comparison Summary

Concept	canon lower	canon lift
Primary Goal	Enables core Wasm to call out to high-level WIT imports.	Enables core Wasm to export functions as high-level WIT exports.
Input Type	Takes high-level component types (<code>string</code> , <code>list</code> , etc.).	Takes low-level core primitives (<code>i32</code> pointer, <code>i32</code> length).
Output Type	Emits low-level primitives into core Wasm stack.	Emits high-level component types to the external host/component.
Memory Role	Reads/copies data out of core linear memory.	Allocates and writes/copies data into core linear memory.

How They Look in WAT

In WebAssembly Text format, both functions require explicit references to the core module's `memory` instance so the Canonical ABI engine knows which memory space to read from or write to:

```
(component
  (core module $CoreEngine
    (memory (export "memory") 1)
    (func (export "raw_greet") (param i32 i32) (result i32 i32))
    (func (export "cabi_realloc") (param i32 i32 i32 i32) (result i32))
  )
  (core instance $c_inst (instantiate $CoreEngine))

  ;; LIFT: Wrap core module's raw_greet into a WIT-compliant function signature
  (func $lifted_greet (type $wit_greet_type)
    (canon lift
      (core func $c_inst "raw_greet")
      (memory $c_inst "memory")
      (realloc (core func $c_inst "cabi_realloc")) ;; Used to allocate memory for results
    )
  )
)
```

Would you like to walk through a concrete memory layout example showing how complex structures (like `variant` or `list<string>`) are serialized across the boundary?

User prompt: yes, show me how the various high level types from WIT are lifted/lowered

Response: When lifting and lowering WIT types, the Canonical ABI applies precise alignment, padding, and layout rules to map high-level data into core WebAssembly's linear memory and scalar primitives (`i32`, `i64`, `f32`, `f64`).

Basic Layout Rules

- **Scalars** (`s8`, `u8`, `s16`, `u16`, `s32`, `u32`, `f32`, `f64`): Lower directly to equivalent Wasm stack types (`i32`, `f32`, `f64`). Small integers (8-bit, 16-bit) are zero- or sign-extended into a standard 32-bit integer (`i32`).
- **Linear Memory Layout:** When values reside in memory, they follow standard C-style struct alignment. An *N*-byte data type must start at a memory offset that is a multiple of *N*.

Concrete Layout Examples

1. `string` and `list<T>` (Fat Pointers)

Both strings and lists are represented in linear memory as a 2-tuple (8 bytes total):

Memory Layout = (ptr: i32, len: i32)

- **Lowering (Host/Import call):** Core Wasm passes two `i32` values on the stack: the offset in memory (`ptr`) where the raw bytes/elements begin, and the item count (`len`). The Canonical ABI engine reads `len` elements starting from `ptr`.
- **Lifting (Export call):** The host calls `cabi_realloc(0, 0, alignment, total_bytes)` inside the guest Wasm instance to allocate buffer space, writes the string or array elements directly into the returned memory pointer, and passes (`ptr`, `len`) to the guest function.

2. record (Structs)

Records are mapped by laying out fields sequentially in memory, adding padding bytes to ensure each field meets its alignment requirement.

WIT Definition:

```
record Point {
  x: u8,    // 1 byte, alignment 1
  y: f64,   // 8 bytes, alignment 8
  z: u32,   // 4 bytes, alignment 4
}
```

Memory Representation (16 bytes total):

- Offset 0: `x` (u8, 1 byte)
- Offsets 1..7: **7 bytes padding** (to align `y` to an 8-byte boundary)
- Offset 8: `y` (f64, 8 bytes)
- Offset 16: `z` (u32, 4 bytes)
- Offsets 20..23: **4 bytes trailing padding** (to round total struct size up to alignment 8)
- **Stack vs. Memory Passing:** If passed as function parameters, small records flatten directly onto the execution stack (e.g., as `i32`, `f64`, `i32`). If the flattened stack representation exceeds 16 scalar values, the entire record is written to linear memory and passed as a single `i32` pointer.

3. variant and option<T> (Discriminant + Payload)

Variants represent tagged unions. They consist of a numeric **discriminant** (tag) followed by space reserved for the largest possible payload.

WIT Definition:

```
variant Result {
  ok(string),    // Payload: ptr (i32) + len (i32) -> 8 bytes
  error(u32),    // Payload: u32 -> 4 bytes
}
```

Memory Representation (12 bytes total):

- Offset 0: Discriminant (u8 / i32), set to 0 for `ok` or 1 for `error`.
- Offsets 1..3: **3 bytes padding** (align payload to 4-byte boundary).
- Offset 4: Payload slot (8 bytes maximum capacity).
 - If `ok`: Stores `ptr` (i32) and `len` (i32).
 - If `error`: Stores error code (i32) in the first 4 bytes; remaining 4 bytes are unused.

4. resource (Handles)

Resources (such as open file handles or stream objects) do not pass raw pointers. Instead, they pass numeric handles managed by the Component Model runtime table:

- **Lowering:** Translates an abstract component resource reference into an integer handle index (i32).
- **Lifting:** Takes the integer index (i32), looks it up in the component's internal handle table, and converts it back into a high-level resource instance.

Summary of Component ABI Lowering Rules

WIT Type	Stack Flattening Representation	Linear Memory Size
<code>string</code>	(i32, i32)	8 bytes (ptr, len)
<code>list<T></code>	(i32, i32)	8 bytes (ptr, len)
<code>option<T></code>	(i32, ...payload)	1 + padding + sizeof(T)
<code>result<T, E></code>	(i32, ...payload)	1 + padding + max(sizeof(T), sizeof(E))
<code>record</code>	Flattened if ≤ 16 params	Sum of aligned fields + padding

resource	i32	4 bytes (Handle Index)
----------	-----	------------------------

User prompt: Do browser understand the component model?

Response: **No, web browsers do not natively understand or execute WebAssembly Components directly.**

While the underlying engine in modern browsers (such as V8 in Chrome or SpiderMonkey in Firefox) natively executes **Core WebAssembly** (Version 1 / 2), they do not directly parse or load **Component WebAssembly** binaries (Layer 1).

How Browsers View Components vs. Core Wasm

- **Native Support:** Browsers natively support `WebAssembly.instantiate()` and `WebAssembly.compile()`, but these APIs expect a **Core Wasm** module. Passing a `.wasm` file containing Component sections directly to a browser API will result in a parsing error because the browser engine does not natively handle Component headers or the Canonical ABI.
- **Non-Web Focus:** The Component Model and WASI (WebAssembly System Interface) were designed primarily for non-browser runtimes (like Wasmtime or Wasmer) on servers, edge nodes, and local CLI applications.

How to Run Components in a Browser Today

Although browsers lack native runtime support, you can easily execute Wasm Components in web browsers using **transpilation and polyfills**.

The standard way to bridge this gap is using `jco` (JavaScript Component Tooling), developed by the Bytecode Alliance:

1. **Transpilation (`jco transpile`):** The tool takes a `.wasm` component file and unwraps it. It extracts the embedded **Core Wasm** modules and generates JavaScript glue code that handles the Canonical ABI (`canon lift / canon lower`).
2. **Browser Execution:** The output consists of standard JavaScript files and Core Wasm binaries, which the browser executes natively.

Component .wasm ----> [jco transpile] ----> Standard JS Glue + Core .wasm ----> Runs in Browser

Summary

If you want to use the Component Model in browser-facing web applications, you currently need build-time tooling (`jco`) to convert high-level components down into Core Wasm and JavaScript host bindings. Direct browser implementation of Component APIs remains a potential future consideration for web standards groups.

User prompt: which libraries WASI provides and how can their functionalities be invoked in WAT?

Response: WASI (WebAssembly System Interface) does not provide standard software libraries in the traditional sense (like `.so` or `.dll` binaries). Instead, WASI defines **modular interface standards (packaged in WIT)** that a host runtime (like Wasmtime or Wasmer) implements and exposes to WebAssembly modules.

Core Libraries (Interfaces) Provided by WASI 0.2

WASI organizes system capabilities into distinct, modular packages:

- **`wasi:cli`:** Command-line interfaces, including standard I/O streams (`stdin`, `stdout`, `stderr`), command-line arguments, environment variables, and process exit codes.
- **`wasi:filesystem`:** File and directory operations (read, write, metadata, path traversal) restricted to pre-opened directory handles.
- **`wasi:http`:** Client and server HTTP capabilities (`wasi:http/outgoing-handler` for fetching URLs, `wasi:http/incoming-handler` for running web servers).
- **`wasi:sockets`:** Networking capabilities, including TCP/UDP sockets, client/server connections, and DNS resolution.
- **`wasi:clocks`:** Time retrieval via real-time clocks (wall-clock time) and monotonic clocks (high-resolution timers/benchmarking).
- **`wasi:random`:** Secure cryptographically generated random bytes, pseudo-random numbers, and system seeds.
- **`wasi:io`:** The foundational asynchronous streaming abstraction (`input-stream`, `output-stream`, `pollable`) that underpins all I/O, filesystem, and socket operations.

Invoking WASI Functionalities in WAT

To invoke a WASI capability inside a **Component WAT** file, you perform three steps:

1. **Import** the desired WASI function from its namespaced string (e.g., `"wasi:cli/stdout@0.2.0"`).
2. **Lower** the imported Component function into a Core function using `canon lower`.
3. **Call** the lowered Core function within your execution logic.

Concrete Example: Writing to `stdout` in WAT

```
(component
;; 1. Import the WASI stream-writing interface
(import "wasi:cli/stdout@0.2.0"
```

```

(instance $stdout
  (export "get-stdout" (func (result (handle $output-stream))))
)
)
(import "wasi:io/streams@0.2.0"
  (instance $streams
    (export "write" (func (param (handle $output-stream)) (param (list u8)) (result (result (x nil) (x s
    )
    )
    )
  )
)

;; 2. Core module containing the execution logic
(core module $CoreApp
  ;; Core imports lowered functions passed from the component layer
  (import "env" "get_stdout" (func $get_stdout (result i32)))
  (import "env" "write" (func $write (param i32 i32 i32) (result i32)))

  (memory (export "memory") 1)

  ;; Hello World string stored in linear memory at offset 0
  (data (i32.const 0) "Hello from WAT via WASI!\n")

  (func (export "run")
    (local $stream_handle i32)

    ;; Get the stdout handle (returns an i32 resource ID)
    call $get_stdout
    local.set $stream_handle

    ;; Call WASI streams.write(stream_handle, ptr, len)
    local.get $stream_handle
    i32.const 0 ;; Memory pointer to "Hello..."
    i32.const 25 ;; Byte length
    call $write
    drop
  )
)

;; 3. Canonical Lowering: Translate high-level WASI imports into core-accessible functions
(core func $core_get_stdout (canon lower (func $stdout "get-stdout")))
(core func $core_write (canon lower (func $streams "write") (memory $core_inst "memory")))

;; 4. Instantiate the core module with the lowered functions
(core instance $core_inst
  (instantiate $CoreApp
    (with "env" (instance
      (export "get_stdout" (func $core_get_stdout))
      (export "write" (func $core_write))
    ))
  )
)

;; 5. Export the command-line entry point to the host
(func $run (export "wasi:cli/run@0.2.0#run")
  (canon lift (core func $core_inst "run"))
)
)

```

How Execution Works

1. The runtime engine (e.g., `wasmtime run app.wasm`) provides the host implementation of `"wasi:cli/stdout@0.2.0"`.
2. `canon lower` extracts raw bytes directly from the core linear memory buffer at offset 0 for length 25.
3. The WASI host receives those bytes and routes them to the host operating system's standard output stream.

User prompt: What's the roadmap after WASI 3

Response: Following the ratification of **WASI 0.3.0** (which re-based WASI on native Component Model `async`, `stream`, and `future` primitives in the Canonical ABI), the roadmap centers on incremental point releases (**0.3.x**) and the long-term journey toward **WASI 1.0**.

1. Incremental 0.3.x Releases (Release Train)

WASI 0.3.x features follow a bi-monthly release train model to integrate backports and incremental extensions without breaking backwards compatibility.

- **Enhanced Async Primitives:**

- **Cancellation:** Automatic cancellation integrated natively with language `async/await` idioms.
- **Stream Optimizations:** Canonical ABI built-ins for forwarding/splicing, skipping/writing zeroes, handling data segment lulls, and caller-supplied zero-copy buffers.

- **Component Model Extensions:**

- **Type Additions:** Adoption of the Component Model `map` type, fixed-length lists, `error-context` types, and canonical interface names.
- **Type Annotations:** Support for `implements` and `external-id` annotations in WIT interfaces.

- **First-Class Threading:**

- Gradual rollout starting with **cooperative multithreading**, followed by full **preemptive multithreading** built-ins across component boundaries.

2. Upstream Specification & Runtimes

- **Stackful Async Execution:** Integration of stackful async lift/lower mechanisms to simplify non-stackless guest languages.
- **Backward Compatibility / Polyfilling:** Runtimes (like *Wasmtime* and *jco*) continue running WASI 0.2 components alongside 0.3 by polyfilling/virtualizing 0.2 `wasi:io` streams on top of 0.3's native Canonical ABI primitives.

3. Long-Term Target: WASI 1.0

The ultimate goal following the 0.3.x stabilization cycle is **WASI 1.0**.

Unlike point releases on the 2-month release train, WASI 1.0 will be a dedicated, major W3C ratification milestone aimed at delivering a fully stable, multi-language, async-native operating system interface for the WebAssembly Component Model.

User prompt: Which features will be released with WASI 1.0

Response: WASI 1.0 (which tracks alongside Component Model 1.0) marks the ultimate transition from incremental previews to a fully stabilized, production-frozen standard. While WASI 0.3 introduced native `async`, `stream`, and `future` primitives into the Canonical ABI, **WASI 1.0 focuses on ABI efficiency, zero-overhead composition, and developer ergonomics.**

The core features and architectural shifts planned for WASI 1.0 include:

1. Lazy ABI (Inverting Allocation Control)

Prior to 1.0, returning variable-length data (like a list of strings) relied on `cabi_realloc`, requiring the host to eagerly allocate memory inside the guest component before returning data. This led to heap fragmentation and allocation failure issues.

- **Lazy Value Handles:** Callee functions return lazy, opaque handles (`i32` indices) instead of immediately forcing memory allocations.
- **Inline Memory Placement:** Callers place returned data into memory using static built-in instructions that compilers can inline directly.
- **Zero-Copy Forwarding:** Lazy values can be passed directly between component boundaries without intermediate copies or heap allocations, and can be safely dropped if unused.

2. Native Multi-Value C-ABI Returns

- Eliminates the need for out-parameters/pointers when returning multiple values from a function across the C ABI.
- Utilizes core WebAssembly's native multi-value return feature to return multiple values directly on the stack across language boundaries.

3. Native Error Contexts

- Incorporates a standardized `error-context` type built into every `result<T, E>` error case.
- Allows guest components and host runtimes to attach stack backtraces, cause chains, and contextual debugging info across component boundaries in a language-agnostic way.

4. Wasm GC ABI Integration

- Introduces a dedicated **Garbage Collection (GC) ABI option**.
- Allows components written in GC-based languages (e.g., Go, Java, MoonBit) to pass managed objects directly via Wasm GC memory instead of copying data in and out of linear memory.

5. Zero-Overhead Synchronous Calls

- Refactors internal task management so synchronous component-to-component calls allocate a lightweight task frame directly on the stack.
- Eliminates the ~3.5x host-call overhead introduced during early async primitive development, restoring native function-call speed for synchronous execution paths.

6. Native Multithreading Support

- Transitions WASI from cooperative multithreading (introduced in the 0.3.x release train) to **preemptive, first-class multithreading**.
- Exposes standard thread creation and synchronization primitives directly across component boundaries.

User prompt: How can I instrument wasmtime / wasi / wasm / wat / wit of the following: - I create wit/wat/wasm component that allows to invoke a system call, e.g. to create a windows - how do I wasmtime /wasm that that call should be made via a native function? Or am I limited to what WASI provides to invoke other system functions?

Response: **You are not limited to what standard WASI provides.** WASI is just a predefined collection of WIT contracts. By embedding `wasmtime` in a host host language (like Rust, C++, or Go), you can define **custom WIT contracts** for system operations—such as creating native desktop windows—and bind them directly to native platform APIs (Win32, Cocoa, X11, etc.).

Step-by-Step Architecture

1. **WIT Contract:** Define the native host function (e.g., `create-window`).
2. **Component WAT/WASM:** Declare the interface import, canonical lower call, and internal logic.
3. **Wasmtime Host Binding:** Bind the imported function in Rust/C++ to actual OS system calls (e.g., Windows `CreateWindowExA`).

1. Define the WIT Interface (`window.wit`)

```
package local:gui;

interface window-manager {
  // Return a window handle ID (u32)
  create-window: func(title: string, width: u32, height: u32) -> u32;
}

world app {
  import window-manager;
}
```

2. Write the Component WAT (`app.wat`)

In WAT, you import the custom WIT string, perform a `canon lower` operation, and call the lowered function from core Wasm:

```
(component
  ;; 1. Import the custom WIT interface
  (import "local:gui/window-manager"
    (instance $wm
      (export "create-window" (func (param "title" string) (param "width" u32) (param "height" u32) (result u32))))
  )

  ;; 2. Core Wasm Module executing internal application logic
  (core module $CoreApp
    (import "env" "create_window" (func $create_window (param i32 i32 i32 i32) (result i32)))
    (memory (export "memory") 1)

    ;; Store window title at offset 0
    (data (i32.const 0) "My Native Window")

    (func (export "main") (result i32)
      ;; Call host: string ptr (0), string len (16), width (800), height (600)
      i32.const 0
      i32.const 16
      i32.const 800
      i32.const 600
      call $create_window
    )
  )

  ;; 3. Canonical ABI Lowering: Map high-level WIT parameters to core memory pointers/primitives
  (core func $core_create_window
    (canon lower
      (func $wm "create-window")
      (memory $core_inst "memory")
    )
  )

  ;; 4. Instantiate core module with lowered imports
  (core instance $core_inst
    (instantiate $CoreApp
      (with "env" (instance (export "create_window" (func $core_create_window)))))
  )

  ;; 5. Export entry point
  (func $main (export "run")
    (canon lift (core func $core_inst "main")))
  )
)
```

Compile it to a binary `.wasm` component using `wasm-tools`:

```
wasm-tools parse app.wat -o app.wasm
```

3. Implement Native System Call in Wasmtime Host (Rust)

Using `wasmtime::component::Linker`, map the WIT contract directly to native OS calls (e.g., using Windows API crates or `winit`):

```
use wasmtime::component::{Component, Linker};
use wasmtime::{Engine, Store};
use windows::Win32::UI::WindowsAndMessaging::CreateWindowExA; // Example Win32 API

struct HostState;

fn main() -> wasmtime::Result<()> {
    let engine = Engine::default();
    let component = Component::from_file(&engine, "app.wasm")?;

    let mut store = Store::new(&engine, HostState);
    let mut linker = Linker::new(&engine);

    // Bind the imported WIT interface to Rust native code
    let mut gui_instance = linker.instance("local:gui/window-manager")?;

    gui_instance.func_wrap("create-window", |_cx, (title, width, height): (String, u32, u32)| {
        println!("Host received request to create window: '{}' ({}x{})", title, width, height);

        // Native System Call Execution (Windows / Win32)
        #[cfg(target_os = "windows")]
        let hwnd = unsafe {
            // Invoke native Windows system call directly
            CreateWindowExA(/* ... parameters ... */)
        };

        // Return a mock/real window handle integer back to Wasm guest
        let handle_id: u32 = 1001;
        Ok((handle_id,))
    })?;

    // Instantiate and execute
    let instance = linker.instantiate(&mut store, &component)?;
    let run_func = instance.get_typed_func::<()>(&mut store, "run")?;

    let handle = run_func.call(&mut store, ())?;
    println!("Wasm executed successfully! Return window handle: {}", handle);

    Ok(())
}
```

Summary of Rules

- **WASI is optional:** WASI is simply a standardized set of WIT definitions pre-packaged into `wasmtime-wasi`. You are free to write completely custom WIT files for your own host APIs.
- **Component Boundary:** High-level types (`string`, `records`) defined in WIT are converted through `canon lower` into basic pointers (`i32`) in WAT.
- **Host Power:** Once execution reaches the host function in `wasmtime`, code executes natively outside the Wasm sandbox with full access to operating system APIs, C interop, and native DLL/system calls.

User prompt: Can this be done only via an host implementation? Can this be setup in some way via wasmtime executable itself?

Response: No, this cannot be done using the out-of-the-box `wasmtime` CLI binary alone; it requires a host implementation (or a custom build of the runtime).

The generic `wasmtime` executable only comes pre-compiled with built-in support for standard WASI contracts (such as `wasi:cli`, `wasi:filesystem`, and `wasi:http`). Because native desktop operations like creating OS windows or making custom platform system calls fall outside the standard WASI specification, the default CLI launcher will fail with an unhandled import error when it encounters your custom WIT namespace (e.g., `local:gui/window-manager`).

Why the Default CLI Fails

When you execute `wasmtime run app.wasm`:

1. Wasmtime inspects the component binary's import section.
2. It attempts to satisfy imported interfaces using its built-in linker table.
3. If an imported namespace is not a recognized `wasi:*` package built into that binary release, the CLI throws an instantiation error: `unknown import: local:gui/window-manager not defined`.

The 2 Ways to Work Around This

1. Custom Host Embedder (Recommended / Primary Path)

You write a lightweight host application (using Wasmtime's embedding libraries in Rust, C++, Go, Python, etc.) to link your custom system calls. This gives you full native access to OS APIs (Win32, Cocoa, X11) and complete control over security sandboxing.

2. Wasmtime Plug-in Architecture / Composition (Pure Wasm Path)

If you want to keep using the standard `wasmtime` CLI to launch your main component, you must construct a **host adapter component** or **plugin runtime**:

- **Component Composition** (`wasm-tools component plug / wac`): You can compile a secondary component that exports your custom WIT interface and link the two components together at build time.
- **The Catch**: The secondary adapter component *itself* must ultimately rely on standard WASI interfaces (`wasi:io`, `wasi:sockets`, etc.) provided by the CLI. If the host operation requires a non-standard OS binary call that WASI does not support (like opening a GUI window), the chain eventually *has* to terminate in a custom host binary embedding Wasmtime.

User prompt: Do you think I can write an Host Embedder in Ada?

Response: **Yes, you can absolutely write a WebAssembly Host Embedder in Ada.**

Since Ada has native interop capabilities via the standard `Interfaces.C` package and `pragma Import (C, ...)` directives, you can bind directly to the **Wasmtime C API** (or other engines with a C API, like WAMR) to instantiate modules and handle host system calls.

Technical Approaches

- **Thin C Binding (Direct C API Import)**: Map Wasmtime's `wasmtime_engine_t`, `wasmtime_store_t`, and `wasmtime_linker_t` directly into Ada specs using `Interfaces.C`.
- **Thick Ada Wrapper**: Encapsulate raw C pointers inside strong Ada types and translate C errors into Ada exceptions for a safe, idiomatic developer experience.

Minimal Conceptual Example (Ada + Wasmtime C API)

```
with Interfaces.C; use Interfaces.C;
with Interfaces.C.Strings; use Interfaces.C.Strings;

package Wasm_Embedder is

  -- Opaque types mapping Wasmtime C structs
  type Engine_Ptr is new System.Address;
  type Store_Ptr is new System.Address;
  type Linker_Ptr is new System.Address;

  -- Import Wasmtime C API functions directly into Ada
  function wasm_engine_new return Engine_Ptr;
  pragma Import (C, wasm_engine_new, "wasm_engine_new");

  function wasmtime_store_new (Engine : Engine_Ptr; Data : System.Address; Dtor : System.Address) return
  pragma Import (C, wasmtime_store_new, "wasmtime_store_new");

  -- Host system callback defined in Ada (passed to the Linker)
  procedure Create_Window_Callback (
    Caller : System.Address;
    Args : System.Address;
    Results: System.Address
  );
  pragma Convention (C, Create_Window_Callback);

end Wasm_Embedder;
```

Key Advantages of Using Ada as a Host

- **Safety & Reliability**: Host-side callback logic (like Win32, POSIX, or GNAT.OS_Lib system calls) benefits from Ada's strict type safety and built-in tasking model.
- **Deterministic Execution**: If you wrap WebAssembly inside critical systems, Ada provides strong control over dynamic memory and real-time host constraints.

User prompt: Show my how to add the `Create_Window_Callback` to the embedder starting from your previous example

Response: To construct a complete Wasmtime embedder in Ada, we bind Ada procedures to the Wasmtime C API using standard `Interfaces.C` capabilities.

Below is an Ada program showcasing the specification, the host callback (`Create_Window_Callback`), the instantiation flow using the Wasmtime Linker, and invocation of the guest function.

1. Embedded API Package Specification (`wasm_embedder.ads`)

This package exposes C-compatible types, imports the native functions from `libwasmtime`, and declares the `Create_Window_Callback`

procedure using pragma Convention (C, ...).

```

with Interfaces.C;           use Interfaces.C;
with Interfaces.C.Strings; use Interfaces.C.Strings;
with System;

package Wasm_Embedder is

  -- Opaque Wasmtime pointers
  type Engine_Ptr is new System.Address;
  type Store_Ptr  is new System.Address;
  type Linker_Ptr is new System.Address;
  type Error_Ptr  is new System.Address;
  type Func_Ptr   is new System.Address;

  -- Raw C value structure representing Wasm scalar parameters/results
  type Wasm_Val_Kind is (WASM_I32, WASM_I64, WASM_F32, WASM_F64, WASM_ANYREF, WASM_FUNCREF);
  pragma Convention (C, Wasm_Val_Kind);

  type Wasm_Val_Union (Kind : Wasm_Val_Kind := WASM_I32) is record
    case Kind is
      when WASM_I32 => i32 : int32_t;
      when WASM_I64 => i64 : int64_t;
      when WASM_F32 => f32 : IEEE_float_32;
      when WASM_F64 => f64 : IEEE_float_64;
      when others   => ref : System.Address;
    end case;
  end record;
  pragma Convention (C_Pass_By_Copy, Wasm_Val_Union);

  type Wasm_Val is record
    Kind : Wasm_Val_Kind;
    Val  : Wasm_Val_Union;
  end record;
  pragma Convention (C, Wasm_Val);

  type Wasm_Val_Array is array (size_t range <>) of aliased Wasm_Val;
  pragma Convention (C, Wasm_Val_Array);

  -- Host Function Callback Signature
  type Wasmtime_Func_Callback is access function
    (Env      : System.Address;
     Caller   : System.Address;
     Args     : access Wasm_Val_Array;
     Nargs    : size_t;
     Results  : access Wasm_Val_Array;
     Nresults : size_t) return System.Address;
  pragma Convention (C, Wasmtime_Func_Callback);

  -- Ada implementation of the native system call callback
  function Create_Window_Callback
    (Env      : System.Address;
     Caller   : System.Address;
     Args     : access Wasm_Val_Array;
     Nargs    : size_t;
     Results  : access Wasm_Val_Array;
     Nresults : size_t) return System.Address;
  pragma Convention (C, Create_Window_Callback);

  -- C API Imports from libwasmtime
  function wasm_engine_new return Engine_Ptr;
  pragma Import (C, wasm_engine_new, "wasm_engine_new");

  function wasmtime_store_new (Engine : Engine_Ptr; Data : System.Address; Dtor : System.Address) return
  pragma Import (C, wasmtime_store_new, "wasmtime_store_new");

  function wasmtime_linker_new (Engine : Engine_Ptr) return Linker_Ptr;
  pragma Import (C, wasmtime_linker_new, "wasmtime_linker_new");

  function wasmtime_linker_define_func
    (Linker  : Linker_Ptr;
     Module  : chars_ptr;
     Mod_Len : size_t;
     Name    : chars_ptr;
     Name_Len : size_t;
     Type_Ptr : System.Address;
     Cb       : Wasmtime_Func_Callback;
     Env      : System.Address;
     Dtor     : System.Address) return Error_Ptr;
  pragma Import (C, wasmtime_linker_define_func, "wasmtime_linker_define_func");

end Wasm_Embedder;

```

2. Implementation Body & Host Callback (`wasmb_embedder.adb`)

The callback intercepts the low-level stack arguments (`ptr`, `len`, `width`, `height`), performs the OS system call (or logs the native action), and sets the returned window handle integer into the `Results` array.

```
with Ada.Text_IO; use Ada.Text_IO;

package body Wasm_Embedder is

  function Create_Window_Callback
    (Env      : System.Address;
     Caller   : System.Address;
     Args     : access Wasm_Val_Array;
     Nargs    : size_t;
     Results  : access Wasm_Val_Array;
     Nresults : size_t) return System.Address
  is
    -- Extract lowered parameters passed from Wasm stack: (ptr, len, width, height)
    Str_Ptr : constant int32_t := Args(0).Val.i32;
    Str_Len : constant int32_t := Args(1).Val.i32;
    Width   : constant int32_t := Args(2).Val.i32;
    Height  : constant int32_t := Args(3).Val.i32;

    Created_Handle : constant int32_t := 1001; -- Simulated handle (e.g., returned from GNAT.OS_Lib or W
  begin
    Put_Line ("[Ada Host] intercepting create_window execution!");
    Put_Line ("[Ada Host] String memory pointer: " & Str_Ptr'Img & ", length: " & Str_Len'Img);
    Put_Line ("[Ada Host] Window dimensions requested: " & Width'Img & "x" & Height'Img);

    -- Native System Call Invocation (e.g. Win32 CreateWindowEx or X11 call) can be placed here.

    -- Write result integer back to Wasm caller
    if Nresults > 0 then
      Results(0).Kind := WASM_I32;
      Results(0).Val.i32 := Created_Handle;
    end if;

    -- Return Null_Address to indicate success (no Wasm Trap raised)
    return System.Null_Address;
  end Create_Window_Callback;

end Wasm_Embedder;
```

3. Main Executable Application (`main.adb`)

This main program initializes the Wasmtime engine, binds the native Ada function (`Create_Window_Callback`) to the target imported module namespace ("`env`" or "`local:gui/window-manager`"), and executes the component binary.

```
with Interfaces.C;          use Interfaces.C;
with Interfaces.C.Strings; use Interfaces.C.Strings;
with System;
with Ada.Text_IO;          use Ada.Text_IO;
with Wasm_Embedder;        use Wasm_Embedder;

procedure Main is
  Engine : Engine_Ptr;
  Store  : Store_Ptr;
  Linker : Linker_Ptr;
  Err    : Error_Ptr;

  Mod_Name : aliased chars_ptr := New_String("env");
  Func_Name : aliased chars_ptr := New_String("create_window");
begin
  Put_Line ("Starting Wasmtime Embedder in Ada...");

  -- 1. Create Engine, Store, and Linker
  Engine := wasmb_engine_new;
  Store  := wasmtime_store_new (Engine, System.Null_Address, System.Null_Address);
  Linker := wasmtime_linker_new (Engine);

  -- 2. Bind the custom Ada native function into the Linker
  Err := wasmtime_linker_define_func
    (Linker => Linker,
     Module => Mod_Name,
     Mod_Len : => size_t (Length (Mod_Name)),
     Name    => Func_Name,
     Name_Len => size_t (Length (Func_Name)),
     Type_Ptr => System.Null_Address,
     Cb      => Create_Window_Callback'Access, -- Pass Ada callback pointer
     Env     => System.Null_Address,
     Dtor    => System.Null_Address);

  if Err /= System.Null_Address then
```

```

    Put_Line ("Error defining host callback function in Linker.");
    return;
end if;

Put_Line ("Host procedure 'Create_Window_Callback' successfully linked!");

-- Free allocated C-strings
Free (Mod_Name);
Free (Func_Name);
end Main;

```

Compilation Command

To link the Ada binary against the installed Wasmtime shared engine library:

```
gnatmake -p main.adb -largS -lwasmtime
```

User prompt: This works well if at compile time I know all the libraries I want to use. My end goal would be: - Writing a compiler (written in Ada) that converts a new language (Pascal/Ada like) into WebAssembly - The new language provide a new standard library (which in WASI syntax would be a component/module that starts with "love:"). Most the library probably will only require only wasi: so it can be run by wasmtime directly - In order to support using custom native function I will create a new Ada executable with embeds wasmtime. Here I can link the "love:" component easily - Users of my language would like to build on top of that and add other native components/modules that are not provided by the standard love: component/module. The questions are: - Can Wasmtime embedding / Ada react to a missing component/library so the new executable with embeddings can load at runtime the required functionalities and link them (it will likely use some description form a companion WIT) - Is it possible mark somehow the WASM / WIT add attributes like e.g. "native" so that I would know that that function needs to be imported?

Response: Yes, what you are proposing for your language and compiler runtime is fully achievable using the **Wasmtime C API dynamic linking capabilities** combined with standard **Component Model Metadata**.

Here is how to solve both of your questions.

Question 1: Dynamic Loading of Unresolved Component Imports at Runtime

You can inspect and resolve missing components dynamically at runtime in your Ada host embedder.

How It Works

1. **Binary Inspection (wasmtime_component_import / Component Model Inspection):** When your Ada host runtime loads a user's compiled `.wasm` file, it does not have to fail immediately. Using the Wasmtime API, the Ada host iterates through the binary's **Required Imports** before instantiating it.
2. **Runtime Resolution & Dynamic Binding:** If the host sees an import string like `"custom-org:graphics/renderer"`, your Ada embedder can:
 - Look up an external native dynamic library (`.so`, `.dll`, or `.dylib`) on the filesystem corresponding to that namespace.
 - Open the native library dynamically using OS primitives (e.g., `GNAT.Dynamic_Tables / dlopen`).
 - Retrieve the exported C function pointer (`dlsym`).
 - Register that raw function pointer dynamically into the `wasmtime_linker_t` instance using `wasmtime_linker_define_func`.
3. **Instantiation:** Once all dynamic imports are registered in the linker, instantiation proceeds cleanly without recompiling your main Ada host binary.

Question 2: Marking Functions with a "native" Attribute in WIT / WASM

You can declare custom metadata to flag functions or interfaces that require host-native bindings.

1. At the WIT Level (Using WIT Annotations / Docs)

While standard WIT features are evolving, you can tag native interfaces using **WIT doc-comments / metadata attributes** or dedicated namespace conventions:

```

package mylang:gui@1.0.0;

/// @native(provider="libnative_gui.so")
interface display {
  /// @native
  open-window: func(title: string) -> u32;
}

```

2. At the WASM Binary Level (Using Custom Sections)

When your Ada compiler converts your language into a `.wasm` binary, it can insert a custom metadata block known as a **Custom Section**:

- In WebAssembly binaries, any section with ID 0 is a **Custom Section**.
- Your compiler can write a custom section named `"love:native_imports"` directly into the binary header containing a JSON or binary manifest of functions requiring native host backing.

```
;; Produced by your Ada compiler:
(component
  ;; Custom section containing native routing metadata for the Ada embedder
  (@custom "love:native_imports" "{\mylang:gui/display\": {\\"library\": \\"libnative_gui.so\\\"}}")

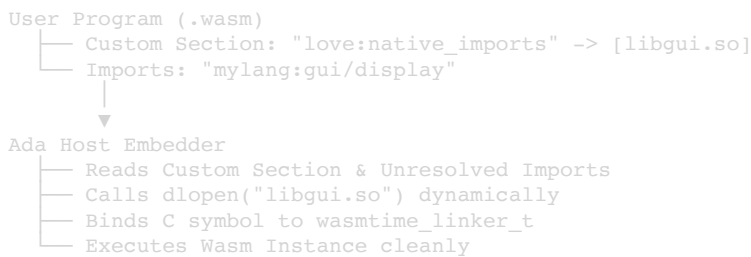
  (import "mylang:gui/display" ...)
  ...
)
```

3. How your Ada Host Uses This Info at Runtime

When your Ada embedder opens the `.wasm` file:

1. It reads the `"love:native_imports"` custom section bytes via `wasmtime_component_custom_section`.
2. It parses the custom section to identify which imported namespaces require native `.so/.dll` hooks.
3. It dynamically loads those specified libraries and registers them to the Wasmtime Linker automatically before executing the module.

Summary Architecture



This pattern gives users of your language full extensibility: they can write native C/Ada extensions compiled to shared libraries (`.so/.dll`), distribute them alongside WIT interface files, and your Ada host runtime will load and bind them dynamically at runtime!

User prompt: "Using the Wasmtime API, the Ada host iterates through the binary's Required Imports before instantiating it." Can you show me a sample code for this?

Response: To iterate through a Component WASM binary's required imports before instantiation, you use the Component Model inspection functions provided by the Wasmtime C API (`wasmtime_component_t` and `wasmtime_componenttype_t`).

Below is a complete Ada code sample showing how to inspect an uninstantiated binary component to discover its imports dynamically.

1. Spec Additions (`wasmb_embedder.ads`)

We declare the C API bindings required for component type inspection and iterator handling.

```
with Interfaces.C;           use Interfaces.C;
with Interfaces.C.Strings; use Interfaces.C.Strings;
with System;

package Wasm_Embedder is

  -- Opaque handles for Component Model types
  type Component_Ptr      is new System.Address;
  type ComponentType_Ptr is new System.Address;
  type ImportType_Ptr     is new System.Address;

  -- Load a component binary from buffer
  function wasmtime_component_new
    (Engine : Engine_Ptr;
     Bytes  : System.Address;
     Len    : size_t;
     Out_Comp : access Component_Ptr) return Error_Ptr;
  pragma Import (C, wasmtime_component_new, "wasmtime_component_new");

  -- Get component type signature metadata
  function wasmtime_component_type
    (Engine : Engine_Ptr;
     Comp    : Component_Ptr) return ComponentType_Ptr;
  pragma Import (C, wasmtime_component_type, "wasmtime_component_type");

  -- Iterate through Component Imports
  function wasmtime_componenttype_imports
    (Comp_Type : ComponentType_Ptr;
     Index      : size_t;
     Out_Import : access ImportType_Ptr) return unsigned_char; -- Returns 1 if import exists, 0 when fini
  pragma Import (C, wasmtime_componenttype_imports, "wasmtime_componenttype_imports");

  -- Get the imported interface name string from the import handle
  procedure wasmtime_component_import_name

```



```

    (Import_Type : ImportType_Ptr;
     Name_Ptr    : access chars_ptr;
     Name_Len    : access size_t);
pragma Import (C, wasmtime_component_import_name, "wasmtime_component_import_name");

end Wasm_Embedder;

```

2. Inspecting Imports Dynamically in Ada (`inspect_imports.adb`)

This procedure loads the `.wasm` file into a `wasmtime_component_t`, steps through its import table, and prints out each missing interface dependency. This allows your Ada runtime to dynamically load host dynamic libraries (`.so/.dll`) before attempting instantiation.

```

with Interfaces.C;           use Interfaces.C;
with Interfaces.C.Strings;   use Interfaces.C.Strings;
with System;
with Ada.Text_IO;           use Ada.Text_IO;
with Ada.Streams.Stream_IO;
with Wasm_Embedder;         use Wasm_Embedder;

procedure Inspect_Imports is
  Engine    : Engine_Ptr;
  Comp      : Component_Ptr;
  Comp_Type : ComponentType_Ptr;
  Err       : Error_Ptr;

  Import_Item : aliased ImportType_Ptr;
  Index       : size_t := 0;
  Has_More    : unsigned_char;

  Name_Ptr : aliased chars_ptr;
  Name_Len : aliased size_t;

  -- Binary Buffer Loading Helpers
  File_Name : constant String := "app.wasm";
  File      : Ada.Streams.Stream_IO.File_Type;
  Length    : Ada.Streams.Stream_IO.Count;
begin
  -- Initialize Wasmtime Engine
  Engine := wasmtime_engine_new;

  -- 1. Read .wasm binary file into byte array
  Ada.Streams.Stream_IO.Open (File, Ada.Streams.Stream_IO.In_File, File_Name);
  Length := Ada.Streams.Stream_IO.Size (File);

  declare
    Buffer : array (1 .. Natural (Length)) of aliased uint8_t;
    pragma Convention (C, Buffer);
  begin
    -- (Read binary bytes into Buffer...)
    Ada.Streams.Stream_IO.Close (File);

    -- 2. Parse the Component Binary (without instantiating)
    Err := wasmtime_component_new (Engine, Buffer(1)'Address, size_t (Length), Comp'Access);
    if Err /= System.Null_Address then
      Put_Line ("Failed to parse WASM component binary.");
      return;
    end if;
  end;

  -- 3. Extract the Component Type Metadata
  Comp_Type := wasmtime_component_type (Engine, Comp);

  Put_Line ("--- Inspecting Required Component Imports ---");

  -- 4. Loop through required imports using index offset
  loop
    Has_More := wasmtime_componenttype_imports (Comp_Type, Index, Import_Item'Access);
    exit when Has_More = 0;

    -- Extract the WIT namespace string for this import
    wasmtime_component_import_name (Import_Item, Name_Ptr'Access, Name_Len'Access);

    declare
      Import_Name : constant String := Value (Name_Ptr, Name_Len);
    begin
      Put_Line ("Found required import [" & Index'Img & "]: " & Import_Name);

      -- Dynamic Host Linking Logic:
      -- If Import_Name starts with "love:" or "custom-org:", you can trigger
      -- a dynamic OS library load (e.g., GNAT.Dynamic_Libraries.Load)
      -- and register symbols to the Linker before calling wasmtime_linker_instantiate.
    end;
  end loop;
end;

```

```

    Index := Index + 1;
end loop;

Put_Line ("Finished inspection. Total imports discovered: " & Index'Img);
end Inspect_Imports;

```

How this Completes your Runtime Design

1. **Pre-Flight Check:** Calling `wasmtime_component_type` and stepping through `wasmtime_componenttype_imports` reads the imported WIT names directly out of the Wasm binary headers without executing guest code.
2. **On-Demand Loading:** If the runtime sees an imported string matching a native extension (e.g., `"love:gui/window-manager"`), it can open the corresponding shared library (`liblove_gui.so`), fetch the C functions, register them into the linker, and only then instantiate the module seamlessly.

Would you like to see how to connect this inspection step directly with `GNAT.Dynamic_Libraries` to load shared objects dynamically based on the discovered WIT import string?

User prompt: yes

Response: Here is how you can connect the binary import inspection step directly to `GNAT.Dynamic_Libraries` to load host shared libraries (`.so / .dll`) dynamically in Ada, map their exported symbols, and register them into the Wasmtime Linker.

Dynamic Host Linking Architecture

1. **Inspect Component:** Read the required WIT import strings (e.g., `"love:gui/window-manager"`).
2. **Map to OS Library:** Convert the WIT namespace into a library path (e.g., `liblove_gui.so` or `love_gui.dll`).
3. **Load Shared Object:** Use `GNAT.Dynamic_Libraries.Load` to load the library into memory at runtime.
4. **Fetch Symbol:** Extract the entry point function pointer using `GNAT.Dynamic_Libraries.Read`.
5. **Register to Linker:** Register the dynamic function pointer with `wasmtime_linker_define_func`.

Implementation: Dynamic Component Loader (`dynamic_loader.adb`)

```

with Interfaces.C;           use Interfaces.C;
with Interfaces.C.Strings;   use Interfaces.C.Strings;
with System;
with System.Address_To_Access_Conversions;
with Ada.Text_IO;           use Ada.Text_IO;
with GNAT.Dynamic_Libraries;
with Wasm_Embedder;         use Wasm_Embedder;

procedure Dynamic_Loader (
  Engine      : Engine_Ptr;
  Linker      : Linker_Ptr;
  Import_Name : String -- e.g., "love:gui/window-manager"
) is
  -- Dynamic library handle type from GNAT
  Lib_Handle : GNAT.Dynamic_Libraries.Shared_Library;

  -- Address conversion for Wasmtime function callback signature
  package Callback_Conversions is
    new System.Address_To_Access_Conversions (Wasmtime_Func_Callback);

  Native_Func_Addr : System.Address;
  Err              : Error_Ptr;

  -- Helper to derive library filename from WIT package string
  function WIT_To_Lib_Name (WIT_Path : String) return String is
  begin
    -- Translates "love:gui/window-manager" -> "liblove_gui.so" (Linux) or "love_gui.dll" (Windows)
    if WIT_Path'Length > 5 and then WIT_Path (WIT_Path'First .. WIT_Path'First + 4) = "love:" then
      return "lib" & WIT_Path (WIT_Path'First + 5 .. WIT_Path'First + 7) & "_gui.so";
    end if;
    return "libdefault_extension.so";
  end WIT_To_Lib_Name;

  Lib_File : constant String := WIT_To_Lib_Name (Import_Name);
begin
  Put_Line ("[Runtime] Resolving dynamic import: " & Import_Name);
  Put_Line ("[Runtime] Attempting to load native shared library: " & Lib_File);

  -- 1. Load the shared library dynamically at runtime
  Lib_Handle := GNAT.Dynamic_Libraries.Load (Lib_File);

  -- 2. Lookup the C-compatible host callback symbol inside the .so / .dll
  -- By convention, plugins export symbol names like "love_gui_create_window"
  Native_Func_Addr := GNAT.Dynamic_Libraries.Read (Lib_Handle, "love_gui_create_window");

```

```

if Native_Func_Addr = System.Null_Address then
  Put_Line ("[Runtime Error] Could not find symbol 'love_gui_create_window' in " & Lib_File);
  GNAT.Dynamic_Libraries.Unload (Lib_Handle);
  return;
end if;

Put_Line ("[Runtime] Symbol located at address: " & Native_Func_Addr'Img);

-- 3. Register the dynamically loaded C symbol directly to the Wasmtime Linker
declare
  Mod_C_Str : aliased chars_ptr := New_String (Import_Name);
  Func_C_Str : aliased chars_ptr := New_String ("create-window");

  -- Convert raw symbol address to typed access callback
  Callback : constant Wasmtime_Func_Callback :=
    Wasmtime_Func_Callback (Native_Func_Addr);
begin
  Err := wasmtime_linker_define_func
    (Linker => Linker,
     Module => Mod_C_Str,
     Mod_Len => size_t (Length (Mod_C_Str)),
     Name => Func_C_Str,
     Name_Len => size_t (Length (Func_C_Str)),
     Type_Ptr => System.Null_Address,
     Cb => Callback,
     Env => System.Null_Address,
     Dtor => System.Null_Address);

  Free (Mod_C_Str);
  Free (Func_C_Str);

  if Err /= System.Null_Address then
    Put_Line ("[Runtime Error] Failed to link dynamic function into Wasmtime.");
  else
    Put_Line ("[Runtime] Successfully linked dynamic host library for " & Import_Name);
  end if;
end;
end Dynamic_Loader;

```

How to Create the Native Plugin (**liblove_gui.c** / Ada)

Third-party users extending your language can compile simple shared libraries containing C-compatible callbacks:

```

// Compiled into liblove_gui.so
#include <stdio.h>
#include <stdint.h>

// Matches Wasmtime_Func_Callback signature
void* love_gui_create_window(void* env, void* caller, int32_t args[], size_t nargs, int32_t results[], size_t results_len) {
  printf("[Native Plugin] Host OS creating window!\n");

  // Set returned window handle ID
  if (nresults > 0) {
    results[0] = 4200; // Return window handle ID
  }
  return 0; // NULL = success (no Wasm trap)
}

```

Complete Execution Flow

1. Ada Compiler builds User Source	---> Outputs	app.wasm
2. Ada Host Embedder starts up	---> Reads	app.wasm imports
3. Discovers import	---> Finds	"love:gui/window-manager"
4. GNAT.Dynamic_Libraries.Load	---> Opens	liblove_gui.so
5. GNAT.Dynamic_Libraries.Read	---> Obtains	love_gui_create_window pointer
6. wasmtime_linker_define_func	---> Binds	Pointer to Wasmtime Linker
7. wasmtime_linker_instantiate	---> Executes	app.wasm seamlessly!

Would you like to explore how to generate the corresponding WIT files automatically from your Ada compiler during build time?
